SPANS-01: Spans & Distributed Tracing Fundamentals

> **Series:** SPANS — Distributed Tracing and Spans | **Notebook:** 1 of 8 | **Created:** December 2025 | **Last Updated:** 10/02/2026

Understanding the Building Blocks of Observability

This notebook introduces distributed tracing concepts and demonstrates how to query span data in Dynatrace using DQL. You'll learn what spans are, how they form traces, and how to explore your distributed systems.

> 🆕 **New Addition (March 2026)**
>
> **Companion Series**
> - **OPIPE** — Processing spans at ingestion (filtering, enrichment, metric extraction)? See **OPIPE-02: Span Processing & Enrichment**
> - **OPLOGS** — For OpenPipeline log processing, see **OPLOGS-01: OpenPipeline Fundamentals**

Table of Contents

1. What is Distributed Tracing?
2. Understanding Spans
3. Span Anatomy
4. Span Kinds
5. Trace Structure

6. Your First Span Query

Prerequisites

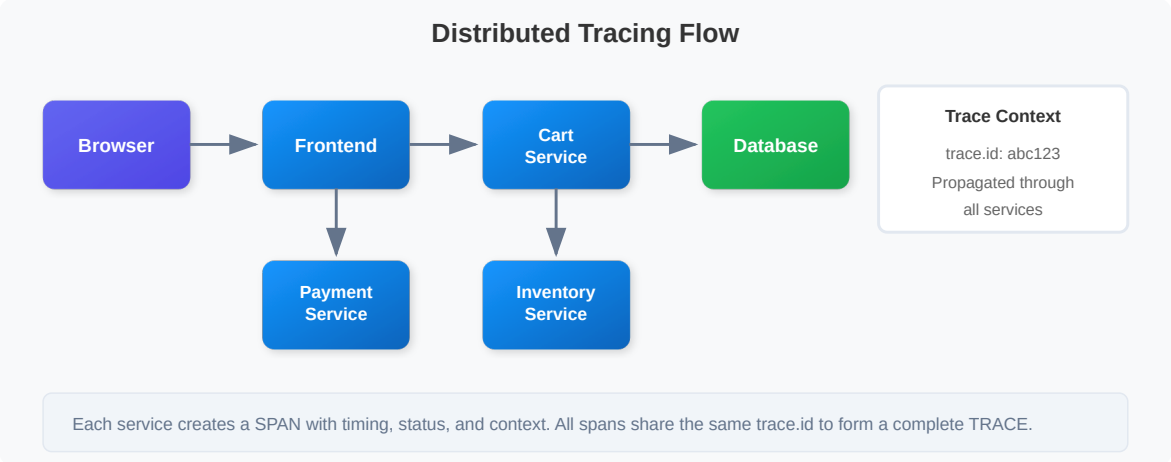
Before starting this notebook, ensure you have:

- ✅ Access to a Dynatrace environment with span data
- ✅ Permission to read spans (`storage:spans:read`)
- ✅ Basic understanding of microservices architecture

1. What is Distributed Tracing?

In modern microservices architectures, a single user request often travels through dozens of services. **Distributed tracing** captures this journey, showing:

- The **path** a request takes through your system
- **Timing** of each operation along the way
- **Dependencies** between services
- Where **errors** and **latency** occur



Without distributed tracing, debugging issues in this flow would require correlating logs from each service manually—an error-prone and time-consuming process.

2. Understanding Spans

A **span** represents a single unit of work in a distributed system. Think of it as a timer that captures:

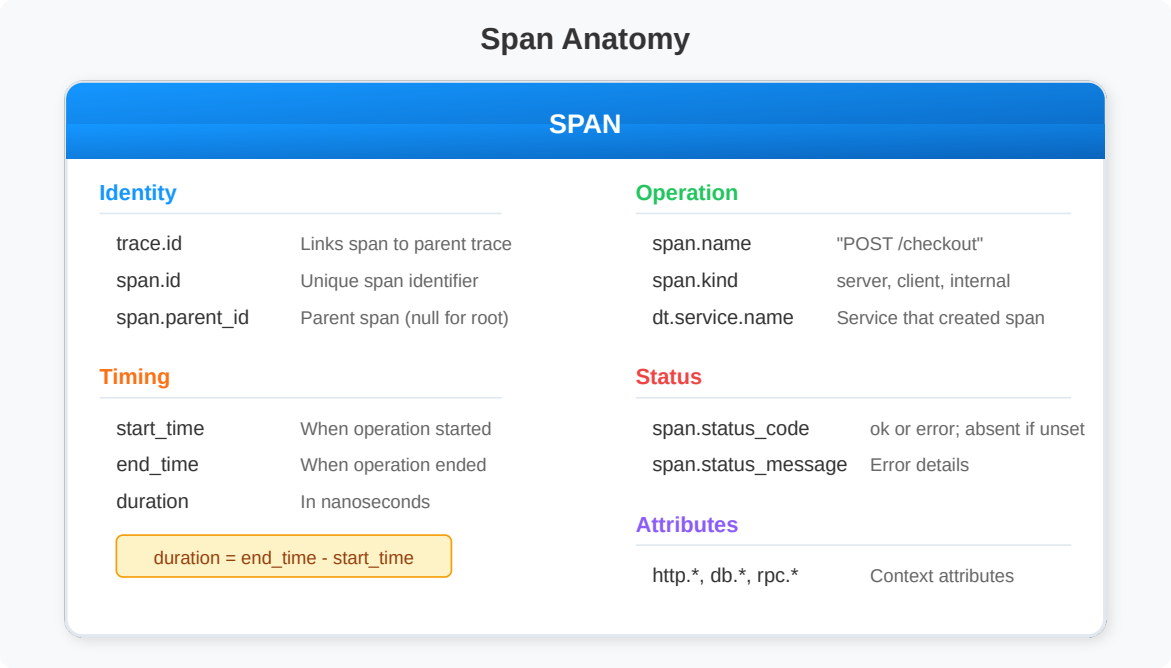
- **What** operation was performed
- **When** it started and ended
- **How long** it took
- **Whether** it succeeded or failed
- **Context** about the operation (HTTP method, database query, etc.)

Key Span Concepts

Concept	Description	Example
Trace	Collection of spans forming a request flow	Checkout transaction
Span	Single operation within a trace	Database query
Root Span	First span in a trace (no parent)	HTTP request to frontend
Child Span	Span triggered by another span	Service calling database

3. Span Anatomy

Every span contains these essential fields:



Core Span Attributes

Attribute	Type	Description
`trace.id`	uid	Unique identifier linking all spans in a trace (16 bytes, hex when shown as a string)
`span.id`	uid	Unique identifier for this specific span (8 bytes)
`span.parent_id`	uid	ID of parent span (null for root spans)
`span.name`	string	Operation name (e.g., "GET /api/products")
`start_time`	timestamp	When the span started
`end_time`	timestamp	When the span ended
`duration`	duration	`end_time` − `start_time`, nanosecond precision — compare it with duration literals (`duration > 100ms`), never bare integers

Service Context Attributes

Attribute	Type	Description
`dt.service.name`	string	Dynatrace service name from service detection — present on every span

Attribute	Type	Description
`dt.smartscape.service`	smartscape ID	Service ID for joins (`dt.entity.service` is the `deprecated` predecessor)
`service.name`	string	Logical service name set by OpenTelemetry SDKs — absent on OneAgent spans
`service.namespace`	string	Optional OpenTelemetry grouping that scopes `service.name` (`experimental`)

> **Which service field?** On the validation tenant, `dt.service.name` was set on all 322,799 spans in an hour and `service.name` on 21,586 (6.7%) — the OpenTelemetry ones. Grouping by `service.name` puts every OneAgent span in one null row, so this series uses `dt.service.name`. Names are not unique, though: on the validation tenant `dt.service.name == "frontend"` covered **4** different services (and `image-provider` 2), from different applications. Where identity matters — error counts per service, dependency maps — group by `dt.smartscape.service`, or by both.

Status Attributes

Attribute	Type	Description
`span.status_code`	string	`ok` or `error` — the field is **absent** when the status is unset, which is most spans
`span.status_message`	string	Optional error text when the status is `error` (`experimental`)

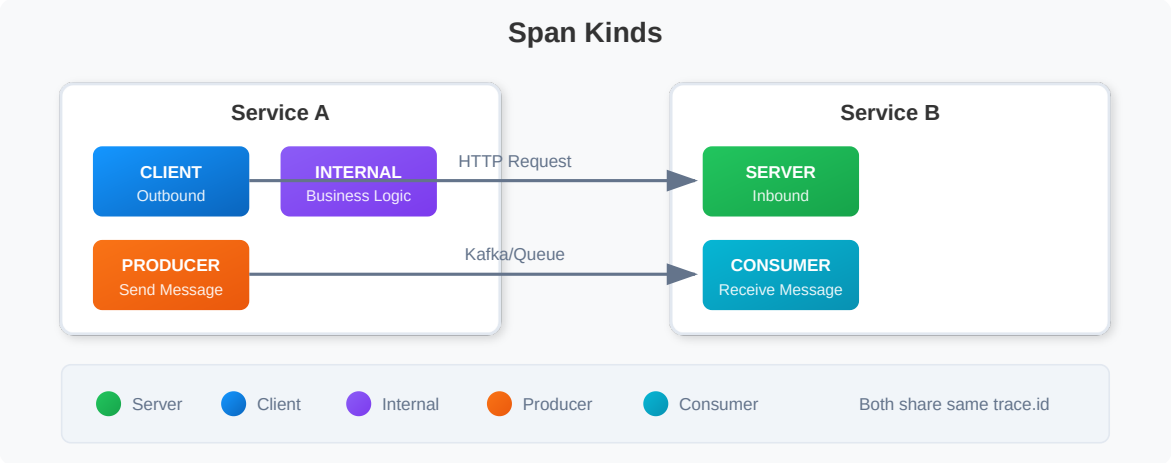
Because unset is stored as a missing field, `span.status_code != "error"` does not count successes — on the validation tenant 307,312 of 318,628 spans in an hour had no status at all, against 240 `ok` and 11,076 `error`. Count failures with `span.status_code == "error"` and derive successes as total minus errors.

> **Sources:** Trace semantic conventions (DT docs) — *"The span status is only present if it is explicitly set to error or ok."* **Dictionary:** `trace.id` / `span.id` / `span.parent_id` (`stable` , `uid`), `duration` (`stable` , `duration`), `span.status_code` (`stable`), `span.status_message` (`experimental`), `dt.service.name` (`stable`), `service.name`

> (`stable`), `service.namespace` (`experimental`), `dt.entity.service` (`deprecated`), read 10/02/2026.

4. Span Kinds

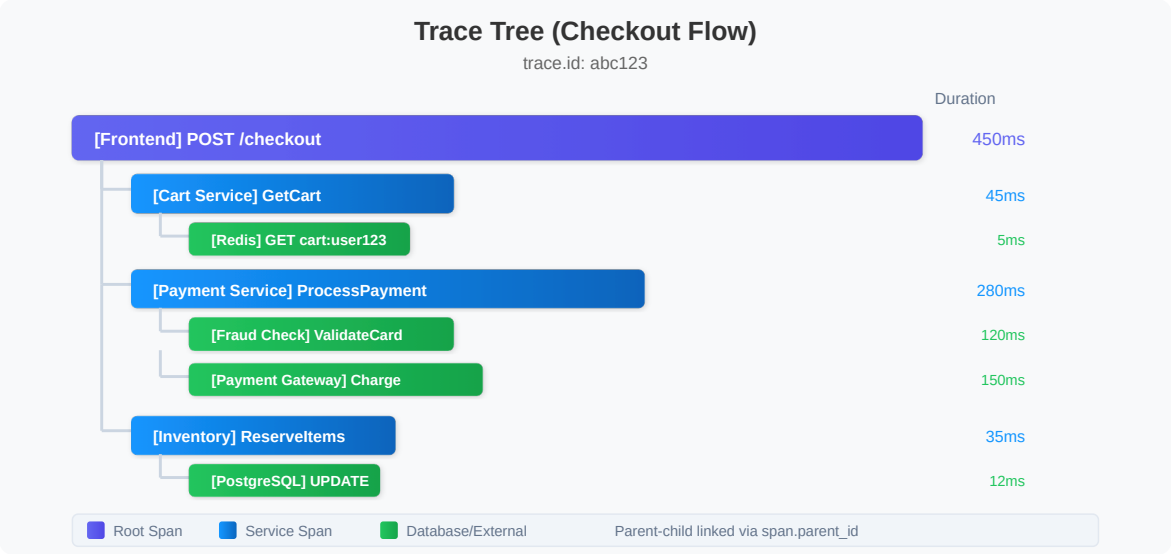
The `span.kind` attribute indicates the span's role in the distributed transaction:



> 💡 **Tip:** When a service calls another service, you'll see a CLIENT span on the caller and a SERVER span on the receiver. Both spans share the same `trace.id`.

5. Trace Structure

A **trace** is a tree of spans connected by parent-child relationships:



Parent-Child Relationships

- Each span (except root) has a `span.parent_id` pointing to its parent
- Root spans have `span.parent_id = null`
- All spans in a trace share the same `trace.id`
- A child span normally starts at or after its parent — but spans from different hosts carry their own clocks, so small skews can make a child appear to start first

6. Your First Span Query

Let's explore span data using DQL. We'll start with the most basic query and progressively add complexity.

> ⚠️ **Important:** Always use `limit` when exploring data to avoid processing millions of spans.

```
// Basic span query - fetch spans from the last hour
fetch spans, from:-1h
| limit 100
```

Selecting Specific Fields

Instead of retrieving all span attributes, select only the fields you need for better performance and readability:

```
// Select specific span fields for analysis
fetch spans, from:-1h
| fields start_time,
         trace.id,
         span.id,
         span.name,
         span.kind,
         dt.service.name,
         duration,
         span.status_code
| sort start_time desc
| limit 50
```

Understanding Duration

`duration` is a **duration** value with nanosecond precision. Divide it by a duration literal to get a plain number in the unit you want:

> 💡 **Tip:** `duration / 1ms` gives milliseconds as a number. `duration / 1000000` does not — dividing a duration by a plain number is still a duration.

```
// Convert duration from nanoseconds to milliseconds and seconds
fetch spans, from:-1h
| fields start_time,
         span.name,
         dt.service.name,
         duration,
         duration_ms = duration / 1ms,     // Convert to milliseconds
         duration_sec = duration / 1s  // Convert to seconds
| sort duration desc
| limit 20
```

Filtering Spans

Use filters to focus on specific spans. Filter early in your query for better performance:

```
// Filter spans by service and span kind
fetch spans, from:-1h
| filter span.kind == "server"
| filter duration &gt; 100ms
| fields start_time,
         dt.service.name,
         span.name,
         duration_ms = duration / 1ms,
         span.status_code
| sort duration_ms desc
| limit 50
```

Discovering Services in Your Environment

Find all services that are generating span data:

```
// Discover all services with span data
fetch spans, from:-1h
| filter span.kind == "server"
| summarize {span_count = count()}, by:{dt.service.name}
| sort span_count desc
| limit 50
```

Summary

In this notebook, you learned:

✅ **What distributed tracing is** and why it's essential for modern architectures
✅ **What spans are** and their core attributes
✅ **Span anatomy** including trace.id, span.id, duration, and status
✅ **Span kinds** (server, client, internal, producer, consumer)
✅ **Trace structure** with parent-child relationships
✅ **Basic DQL queries** to fetch and explore span data
✅ **Duration conversion** with duration literals (`duration / 1ms`)

Next Steps

Continue to **SPANS-02: Querying Spans with DQL** to learn:
- Filtering spans by service, operation, and attributes
- Finding specific traces by trace.id
- Querying HTTP and database spans
- Combining multiple filters for precise analysis

References

- Trace semantic conventions (DT docs)
- Distributed traces (DT docs)
- Traces (opentelemetry.io)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.